PBL PROJECT GUIDE

Server Sync: How the Project Works

From double-clicking `server.exe` to a finished web page - which file runs first, where control goes next, and what happens to the data on the way. Four complete worked examples with real code and real SQL.

1 The overview in 60 seconds
2 Part A - Start-up: what runs first
3 Part B - The thread pool
4 Part C - Example 1: viewing products
5 Part D - Example 2: signing in
6 Part E - Example 3: add to cart and place an order
7 Part F - Example 4: admin cancels an order
8 Part G - The template engine, line by line
9 Part H - Two shoppers, one last item
10 Part I - Every file: when it runs, who calls it
11 Part J - Call-chain cheat sheet
12 Part K - Where to look when something breaks
13 Part L - Watching it live: Server logs
14 Part M - Demo delivery and cancel

Graphic Era PBL project · C++17 · Winsock · MySQL 8 · no web framework.
File paths in this guide are relative to the project folder (`D:\pdb`).

1. The overview in 60 seconds

Server Sync is a small web server that **you wrote yourself**. When you run `server.exe` it opens a door (port 8080) on your computer. A browser knocks, sends a text message (an HTTP request), and the server sends back a text message (an HTML page). Everything else in the project exists to decide *what page to send back*.

A restaurant picture to keep in mind

Door + waiters	Winsock socket + **worker threads** (`core/server.cpp`) - they take the customer's order slip.
Reading the order slip	`core/http.cpp` - turns raw text into a `Request` object.
Front-desk manager	`app.cpp` + `routes.cpp` - who is this customer? which department handles this order?
Department heads	`handlers/` - one function per page; they coordinate but do not cook.
Kitchen	`services/` - the business rules and **all the SQL**.
Pantry	MySQL database (`db/database.cpp` is the only door to it).
Menu cards / plating	`frontend/templates/*.html` + `core/template.cpp` - the same data, dressed as a page.

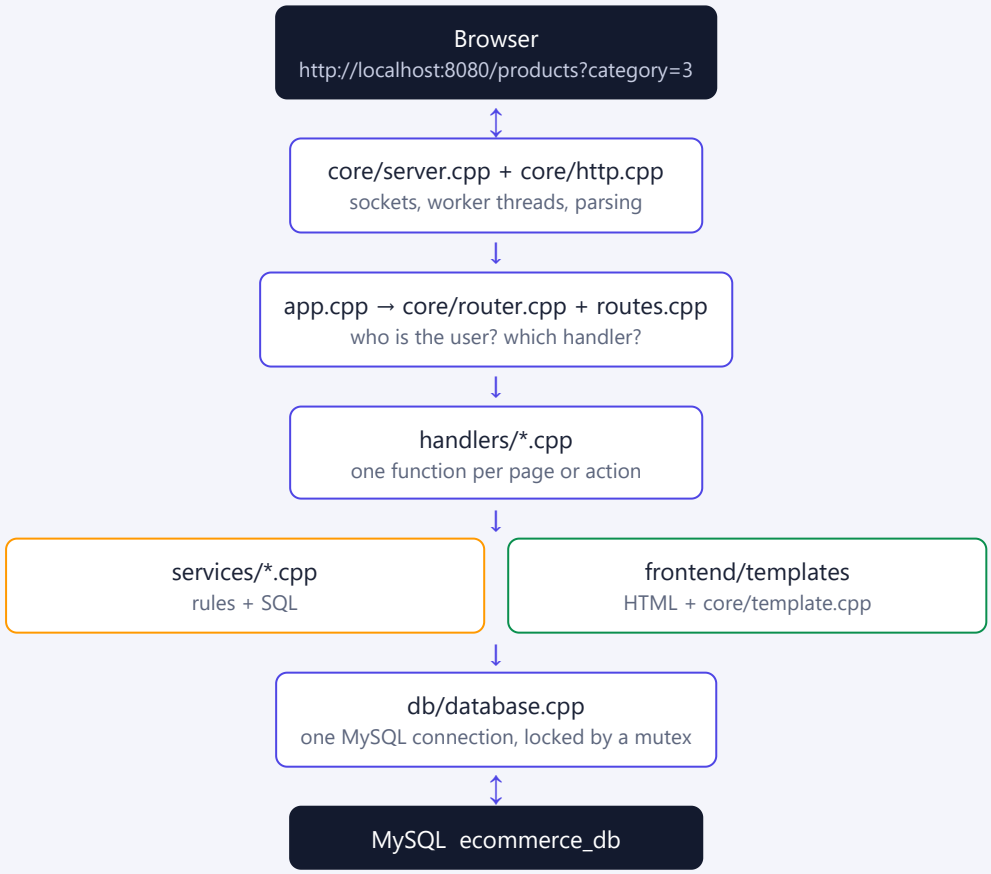


Figure 1 - the layers. A request goes down; the finished page comes back up the same way.

The golden rules of this code base

1. **SQL lives only in `services/`.** Handlers never write SQL.
2. **HTML lives only in `frontend/`.** C++ never builds HTML strings.
3. **URLs live only in `routes.cpp`.** One line per URL, and it says who may use it.
4. **Every function has one job:** *core* knows the web, *services* know the shop, *handlers* connect the two.

The project's files at a glance

Folder	Job
`backend/main.cpp`	The first function that runs (`main()`). Starts everything.
`backend/config.*`	Port number, DB login, low-stock level.
`backend/core/`	Generic web plumbing: `server` (sockets+threads), `http` (parse/build), `router`, `template`.
`backend/app.cpp`, `routes.cpp`	Per-request pipeline and the URL map.
`backend/handlers/`	Page functions: `shop_`, `auth_`, `cart_`, `order_`, `admin_*`, plus `view.cpp` helpers.
`backend/services/`	Product, category, cart, order, user, admin logic and SQL.
`backend/db/database.*`	MySQL connection, `run()/rows()/quote()`, `Transaction`.
`frontend/templates/`	The HTML pages and partials. `frontend/static/` = CSS and JavaScript.
`database/*.sql`	Schema and sample data (you run these in MySQL, not the server).

Part A - Start-up: what runs first

You double-click `run.bat` (or type `.\server.exe`). Windows loads the program and control begins at `main()` in `backend/main.cpp`. Just before that, C++ creates the program's global objects - two matter here:

- `db/database.cpp` `Database db;` - the one shared database object (not connected yet).
- `app.cpp` `static const Router router = buildRoutes();` - this calls `routes.cpp` `buildRoutes()`, which fills the router's table with every URL of the site, **before main() even starts**.

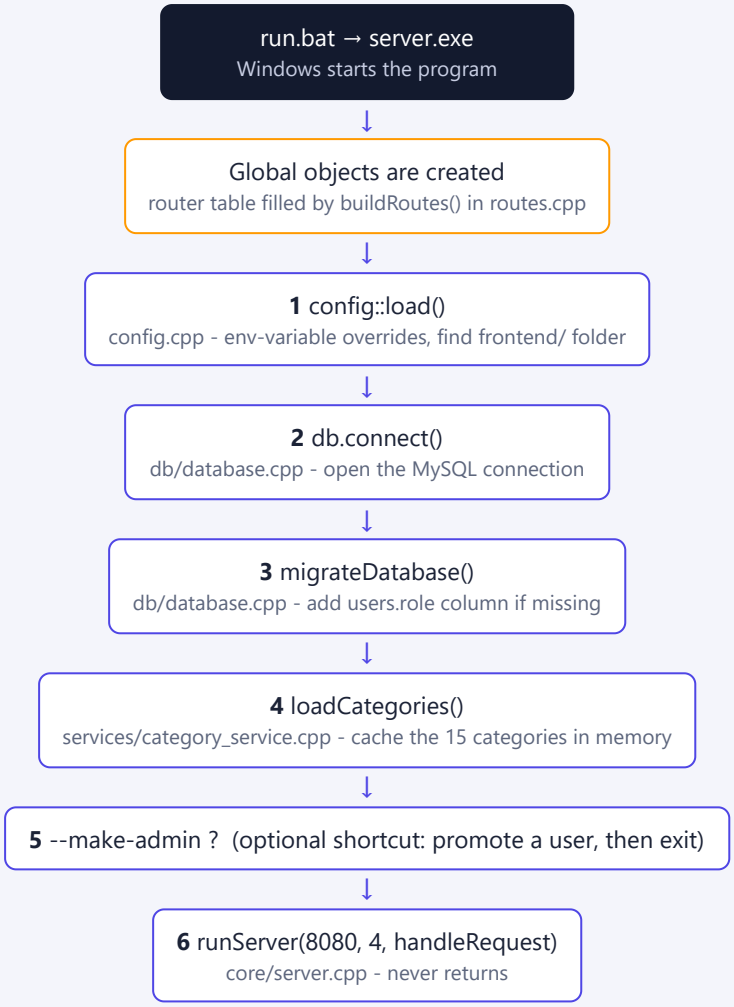


Figure 2 - start-up order. Every step must succeed or the program prints a message and exits.

main() - the whole start-up in one screen

```
int main(int argc, char** argv)
{
    config::load();                       // 1. settings

    if (!db.connect())       return 1;    // 2. database
    if (!migrateDatabase())  return 1;    // 3. schema upgrade
    loadCategories();                     // 4. category cache

    if (argc == 3 && string(argv[1]) == "--make-admin") {      // 5. one-off command
        makeAdminByEmail(argv[2]);  ...  return 0;
    }
    if (countAdmins() == 0)  cout << "[Admin] No admin account yet ...";

    return runServer(config::serverPort, config::workerThreads, handleRequest);  // 6. serve forever
}
```

1 config::load()

`backend/config.cpp`

Defaults live in variables (`dbHost="localhost"` , `dbUser="root"` , `serverPort=8080` , `workerThreads=4` , `lowStockThreshold=5`). Then it reads environment variables such as `ECOM_DB_PASS` - if you set one it overrides the default, so you never have to edit the password in code. Finally it looks for a folder containing `templates/layout.html` (tries `frontend` , `../frontend` , `../../frontend`) and remembers it in `config::frontendDir` . That is why the server finds its HTML no matter where it is started from - and why starting it from the wrong folder shows "*missing template*".

2 db.connect()

`backend/db/database.cpp`

Calls `mysql_init` then `mysql_real_connect(host, user, password, "ecommerce_db", 3306)` from the MySQL C library (`libmysql.dll`), and selects UTF-8. On success it prints `[DB] Connected to MySQL successfully.` If it fails you see `Connection failed: ...` - almost always MySQL is not running or the password is wrong.

3 migrateDatabase()

`backend/db/database.cpp`

Asks `information_schema` whether `users.role` exists. If not, it runs `ALTER TABLE users ADD COLUMN role VARCHAR(10) NOT NULL DEFAULT 'CUSTOMER'` and prints "*Added users.role column*". This is how the admin feature gets its column without you touching SQL.

4 loadCategories()

`backend/services/category_service.cpp`

Runs `SELECT category_id, category_name FROM categories` once and keeps the 15 rows in a `vector<Category>` protected by its own mutex. Why? Every page's header shows the category menu; reading it from memory is faster than querying on every request. The admin's category pages call `loadCategories()` again after any change, so the menu is always current.

5 runServer(port, workers, handleRequest)

`backend/core/server.cpp`

Notice what is passed: **a function**, `handleRequest` (defined in `app.cpp`). `server.cpp` does not know anything about shops - it only knows "when a request arrives, call this function and send back whatever it returns". That is what keeps `core/` reusable. Inside it:

1. `WSAStartup` - turns on Windows sockets.
2. `socket()` , `bind()` to port 8080, `listen()` - open the door. (`Bind failed` means another copy already owns port 8080.)
3. Prints `Server running on http://localhost:8080` .
4. Starts 4 worker threads, all running the function `worker()` .
5. Enters an endless loop: `accept()` a connection → put it on a queue → wake a worker. This is the main thread's only job from now on.

What you see in the console after a healthy start

```
[DB] Connected to MySQL successfully.
Server running on http://localhost:8080
```

Part B - The thread pool: how many users are served

After start-up the program has **five threads**: the main thread (accepts connections) and four workers (handle them). They cooperate through a queue.

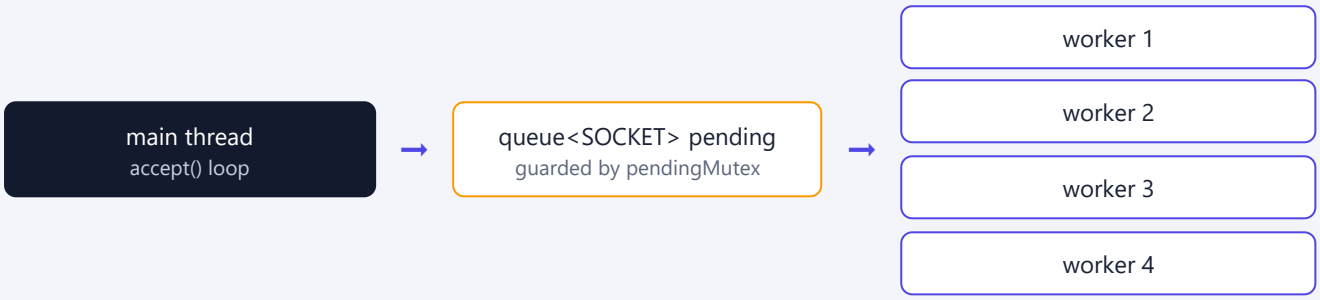


Figure 3 - the main thread pushes each new connection; `condition_variable::notify_one()` wakes exactly one sleeping worker.

The accept loop (main thread)

```
while (true) {
    SOCKET client = accept(listener, nullptr, nullptr);   // sleeps here until a browser connects
    { lock_guard<mutex> lock(pendingMutex);  pending.push(client); }   // put it on the queue
    pendingReady.notify_one();                                  // wake one worker
}
```

What a worker does (the same loop, four times in parallel)

```
static void worker(int id, RequestHandler handler)
{
    while (true) {
        SOCKET client;
        { unique_lock<mutex> lock(pendingMutex);
          pendingReady.wait(lock, []{ return !pending.empty(); });   // sleep until there is work
          client = pending.front();  pending.pop(); }

        string raw;  Request req;
        if (!readRequest(client, raw) || !parseRequest(raw, req)) { closesocket(client); continue; }

        cout << "[W" << id << "] " << req.method << " " << req.path << endl;   // the console log line

        Response res;
        try { res = handler(req); }                      // <-- handler is handleRequest from app.cpp
        catch (const exception& e) { res = Response::html("<h1>500 ...</h1>", 500); }

        sendAll(client, buildHttpResponse(res));
        closesocket(client);                             // one request per connection
    }
}
```

- `readRequest()` keeps calling `recv()` until it has seen the blank line that ends the headers, then reads `Content-Length` more bytes (the form data of a POST). It refuses requests over 64 KB of headers or 1 MB of body.
- The `try/catch` means a bug in one page produces a 500 error page instead of crashing the whole server.
- The console line `[W2] GET /products` tells you **which worker** took **which request** - useful for seeing concurrency in action.

> **Why a pool and not one thread per request?**
> Creating a thread per request is wasteful and unbounded. A fixed pool of 4 reuses threads and caps resource use. If all 4 are busy, new connections simply wait in the queue.

Part C - Example 1: a guest views Audio products

We follow one concrete request from keyboard to screen. The user (not signed in) clicks **Audio** in the menu and sorts by price. The browser requests:

```
GET /products?category=3&sort=price_asc HTTP/1.1
Host: localhost:8080
Accept: text/html
```

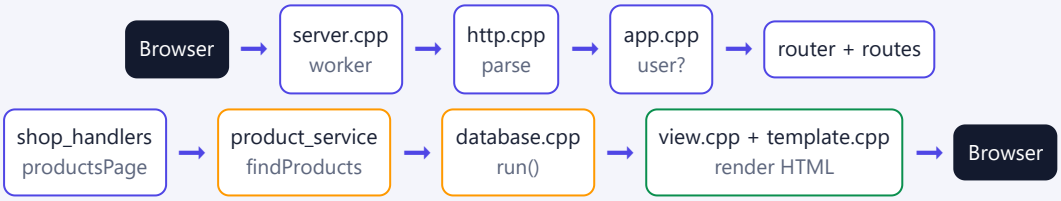


Figure 4 - the two halves of one request: in (top row) and out (bottom row).

1 A worker picks up the connection and reads the bytes

`core/server.cpp` `worker()` → `readRequest()`

The worker that woke up receives the text above. Then it prints `[W3] GET /products` to the console.

2 The text is turned into a Request object

`core/http.cpp` `parseRequest()`

It splits the first line into method / target, separates path from query string, decodes URL escapes (`%20`, `+`), and splits `a=1&b=2` into a map. Result, as a C++ object:

```
Request {
  method  = "GET"
  path    = "/products"
  query   = { "category" -> "3",  "sort" -> "price_asc" }
  form    = { }            // no body in a GET
  cookies = { }            // browser sent none - this is a guest
  user    = { id=0 }       // filled in by the next step
}
```

3 handleRequest(): static file? who is the user?

`app.cpp`

```
Response handleRequest(Request& req) {
    Response res;
    if (serveStatic(req, res))    return res;      // 1. /static/css/... -> just send the file
    identifyUser(req);                             // 2. cookie -> user (guest here: no cookie)
    if (!router.dispatch(req, res)) return notFoundPage(req);   // 3. find the handler
    return res;
}
```

Our path does not start with `/static/`. `identifyUser` finds no `session_id` cookie, so `req.user.id` stays 0 (a guest) and the cart badge stays 0.

4 The router finds the handler

`core/router.cpp` ← `routes.cpp`

The router keeps a map whose keys look like `"GET /products"`. `dispatch()` builds the key from the request (`method + " " + path`) and looks it up. The entry was created by this line in `routes.cpp`:

```
r.get("/products", productsPage);        // no requireLogin: anyone may browse
```

So control now jumps to `productsPage(req)`.

5 The handler reads the filters

`handlers/shop_handlers.cpp` `productsPage()`

```
ProductFilter filter;
filter.categoryId = req.intParam("category", 0);   // -> 3
filter.sort       = req.param("sort");             // -> "price_asc"
Rows products = findProducts(filter);              // ask the "kitchen"
```

`req.param()` looks in the form first, then the query string. The handler does **no SQL** itself - it hands a description of what it wants to the service.

6 The service builds and runs the SQL

`services/product_service.cpp` `findProducts()`

It starts from a base `SELECT ... JOIN categories`, then appends a condition for every filter that is set, and an `ORDER BY` chosen from a fixed list (so users cannot inject their own ordering). For our request it produces:

```
SELECT p.product_id, p.name, p.description, p.price, p.stock, p.category_id, c.category_name
FROM products p JOIN categories c ON p.category_id = c.category_id
WHERE 1=1 AND p.category_id=3
ORDER BY p.price ASC, p.product_id
```

(Free-text such as a search word would be wrapped in `db.quote()`, which escapes quotes - that is the SQL-injection protection.)

7 database.cpp executes it and returns rows

`db/database.cpp` `Database::run()`

It locks the database mutex (only one thread may use the single connection at a time), calls `mysql_query`, stores the result, remembers the column names, and copies every row into a `map<string,string>` (NULL becomes an empty string). The service gets back:

```
Rows = [
  { product_id:"12", name:"Neckband Earphones",      price:"999.00",  stock:"30",
category_name:"Audio", ... },
  { product_id:"9",  name:"TWS Wireless Earbuds",    price:"1499.00", stock:"38", ... },
  { product_id:"10", name:"Bluetooth Party Speaker", price:"2499.00", stock:"18", ... },
  { product_id:"11", name:"ANC Headphones",          price:"3499.00", stock:"15", ... } ]
```

8 Rows become template Data

`handlers/view.cpp` `productData()`, `addProducts()`

For each row the helper creates a `Data` object with display-ready values: `price` formatted as `"₹999"` (Indian digit grouping by `money()`), `icon` = the headphone emoji, `tint` = a CSS gradient chosen from the category id, and flags like `soldOut` / `lowStock`. The handler also adds the sidebar category links, the sort links and the heading *"Audio"*.

9 renderPage(): data + HTML templates = a page

`handlers/view.cpp` `renderPage()` → `core/template.cpp` `renderTemplate()` → `frontend/templates/...`

1. `baseData(req)` adds values every page needs: `loggedIn`, `userName`, `cartCount`, the category list for the menu, the search box text, and the flash message (if the URL had `?notice=...`).
2. The page's own values are merged on top.
3. The content template `pages/products.html` is rendered. Where it says `{{#each products}}{{> partials/product_card.html}}{{/each}}` the engine repeats the product-card file once per product.
4. That finished block becomes `{{{content}}}` inside `layout.html`, which also pulls in `partials/header.html` and `footer.html` (and the login pop-ups, because this visitor is a guest).

10 The response goes out

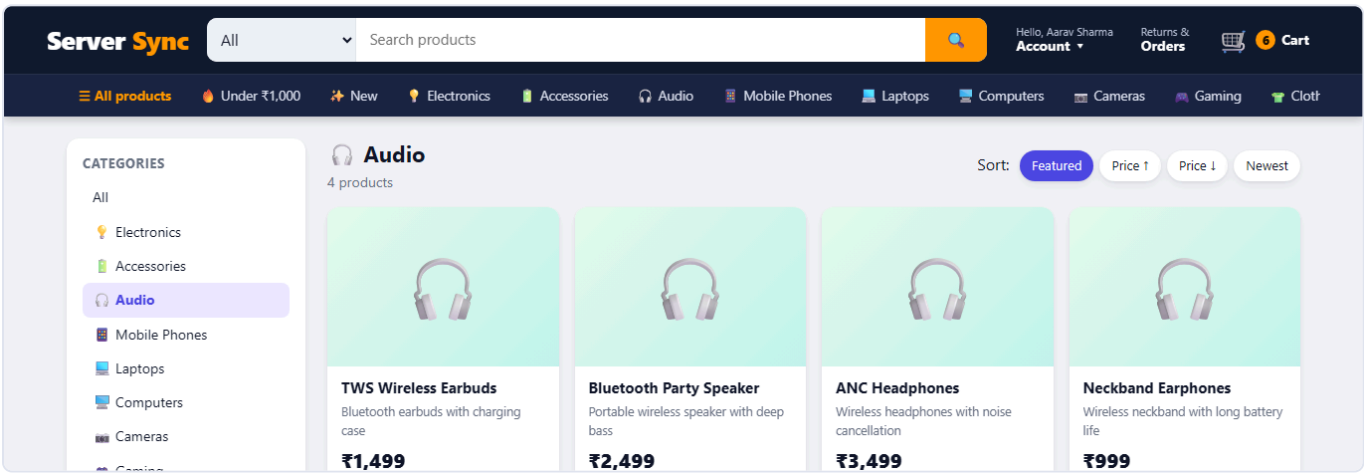
`core/http.cpp` `buildHttpResponse()` → `core/server.cpp`

The handler's `Response::html(page)` becomes bytes, `sendAll()` writes them to the socket, and `closesocket()` ends the conversation:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 9649
X-Content-Type-Options: nosniff
Connection: close

<!DOCTYPE html><html lang="en">...<a class="cname" href="/product?id=12">Neckband Earphones</a>...
```

The browser then notices `<link rel="stylesheet" href="/static/css/style.css">` and makes **more requests**. Those start with `/static/`, so `app.cpp` answers them directly from `frontend/static/` without any handler or database (this is also where `..` in a path is rejected).



The page produced by this request (Audio category).

Part D - Example 2: signing in

This example shows two things: how a **POST form** is handled, and how the server remembers who you are **between requests** (HTTP itself has no memory).

D1. Submitting the form

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 40

email=aarav%40example.com&password=demo123
```

1 Parsing

`core/http.cpp`

`parseRequest()` runs the same pair-splitting on the **body** and fills `req.form = { email:"aarav@example.com", password:"demo123" }` (the `%40` is decoded to `@`).

2 Routing

`routes.cpp`

```
r.post("/login", loginSubmit);        // key "POST /login"
```

The same path `/login` has two entries: `GET` shows the form, `POST` processes it.

3 The handler asks the service to check the password

`handlers/auth_handlers.cpp` `loginSubmit()`

```
User user;
string error = logIn(req.param("email"), req.param("password"), user);
if (!error.empty()) { ... re-show the login page with the error ... }
```

4 logIn() verifies the password

`services/user_service.cpp`

1. Looks up the user: `SELECT user_id, name, email, password, COALESCE(role,'CUSTOMER') AS role FROM users WHERE email='aarav@example.com'` (no row → "*No account found*").
2. The stored password is not the password. It is `sha256$<salt>$<hash>`. `passwordMatches()` splits out the salt, then asks MySQL to compute `SELECT SHA2(CONCAT('<salt>','demo123'),256)` and compares it with the stored hash.
3. If it differs → "*Incorrect password.*" If it matches, the `User` object is filled in.
4. **Legacy upgrade:** if the stored value was an old plain-text password, it is replaced with a salted hash right now.

5 A session is created

`services/user_service.cpp` `startSession()`

It builds a 32-character random id with `rand_s()` (the operating system's random generator - not predictable) and stores it:

```
INSERT INTO user_sessions (session_id, user_id) VALUES ('69e9e7efd1bb087a7f010ff3e5a34422', 8)
```

6 The reply: redirect + cookie

`handlers/auth_handlers.cpp`

```
Response res = Response::redirect(user.role == "ADMIN" ? "/admin" : "/?notice=login");
res.setCookie("session_id", sessionId);
return res;
```

```
HTTP/1.1 302 Found
Location: /?notice=login
Set-Cookie: session_id=69e9e7ef...; Path=/; HttpOnly; SameSite=Lax
```

A **302** tells the browser "go to this other URL". We redirect instead of showing a page so that pressing Refresh cannot re-submit the password form (the *Post/Redirect/Get* pattern). `HttpOnly` hides the cookie from JavaScript; `SameSite=Lax` stops other websites from making your browser send it on their forms.

D2. The next request - "who am I?"

The browser follows the redirect and automatically adds the cookie:

```
GET /?notice=login HTTP/1.1
Cookie: session_id=69e9e7efd1bb087a7f010ff3e5a34422
```



Figure 5 - runs at the start of **every** request that carries a cookie.

```
SELECT u.user_id, u.name, u.email, COALESCE(u.role,'CUSTOMER') AS role
FROM user_sessions s JOIN users u ON s.user_id=u.user_id
WHERE s.session_id='69e9e7ef...' AND s.is_active=TRUE;

UPDATE user_sessions SET last_activity=CURRENT_TIMESTAMP WHERE session_id='69e9e7ef...';
```

From now on `req.user.name` is "Aarav Sharma", the header shows "*Hello, Aarav Sharma*", and the cart badge shows the item count. **Signing out** (`GET /logout` → `endSession()`) sets `is_active=FALSE` and clears the cookie - the stored id is then useless even if someone copied it. An admin can also end any session from the Sessions page.

Two protective wrappers: requireLogin and requireAdmin

`core/router.cpp` - they wrap a handler. `requireLogin(cartPage)` returns a new function that first checks `req.user.loggedIn()` ; if false it redirects to `/login?notice=needlogin` and **never calls** `cartPage` . `requireAdmin` additionally checks `req.user.isAdmin()` and otherwise redirects to `/?notice=forbidden` . Because the wrapper sits in `routes.cpp` next to the URL, you can see every page's access rule in one place.

Part E - Example 3: add to cart, then place an order

This is the heart of the project. Aarav (signed in) opens **ANC Headphones** (product 11, ₹3,499, stock 15), chooses quantity **2** and clicks **Add to Cart**, then checks out.

E1. POST /cart/add

```
POST /cart/add          Cookie: session_id=69e9...
product_id=11&quantity=2
```

1 Routing with a guard

`routes.cpp`

```
r.post("/cart/add", requireLogin(cartAdd));
```

The guard passes (we have a cookie), so `cartAdd` runs.

2 Handler

`handlers/cart_handlers.cpp`

```
string error = addToCart(req.user.id, req.intParam("product_id"), req.intParam("quantity", 1));
if (!error.empty()) return messagePage(req, "Could not add to cart", error, "/cart", "Back to cart");
return Response::redirect(req.param("buy").empty() ? "/cart?notice=added" : "/checkout");
```

On success it redirects. "Buy Now" sends an extra `buy=1` field so the shopper skips the cart page.

3 Service: addToCart() - the rules

`services/cart_service.cpp`

All inside one `DbLock` so nothing changes between the checks and the write:

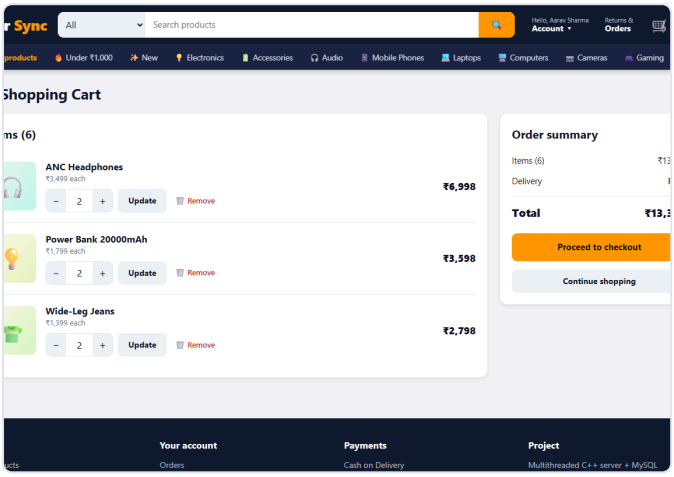
1. `quantity > 0` ? 2 - yes.
2. `stockOf(11)` → `SELECT stock FROM products WHERE product_id=11` → **15**.
3. `activeCartId(userId, true)` - finds the user's `ACTIVE` cart or creates one (`INSERT INTO cart (user_id,status) VALUES (5,'ACTIVE')` → cart id 8).
4. `quantityInCart()` - already in the cart? → 0.
5. Check `already + quantity > stock` ? 0 + 2 > 15 - no, fine.
6. Write: `INSERT INTO cart_items (cart_id, product_id, quantity) VALUES (8, 11, 2)` (if the product were already in the cart it would `UPDATE ... SET quantity=quantity+2` instead - the table has `UNIQUE(cart_id, product_id)`).

Returns `""` (no error) → the handler redirects to `/cart?notice=added` .

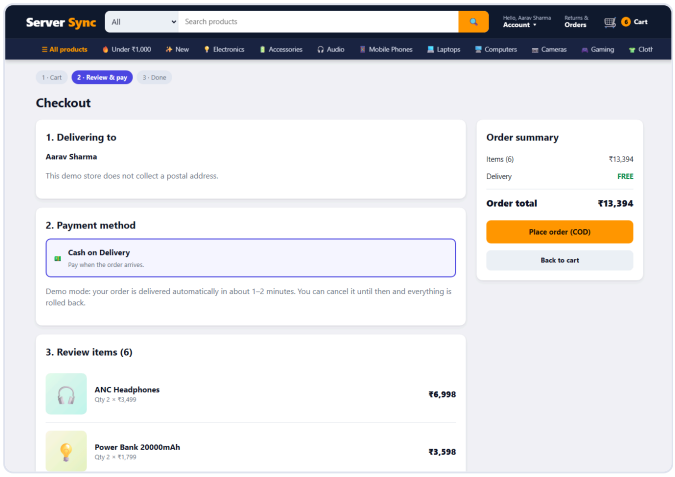
4 **The browser follows the redirect: GET /cart**

`handlers/cart_handlers.cpp` `cartPage()`

Runs `cartItems(userId)` (a join of `cart_items`, `products`, `categories`), builds one `Data` per line, and computes `units` = 2 and `total` = `money(6998)` = **₹6,998**. The green banner "Item added to your cart" appears because the URL has `notice=added` and `view.cpp` looks that code up in a fixed table (so a crafted link can never display arbitrary text).



/cart - lines, quantity steppers, order summary.



/checkout - review and Cash on Delivery.

E2. POST /checkout/place - the transaction

Clicking **Place order** sends `POST /checkout/place`. The handler `order_handlers.cpp` `placeOrderSubmit()` calls `services/order_service.cpp` `placeOrder(userId)`. This is the most important function in the project, so every step is listed.

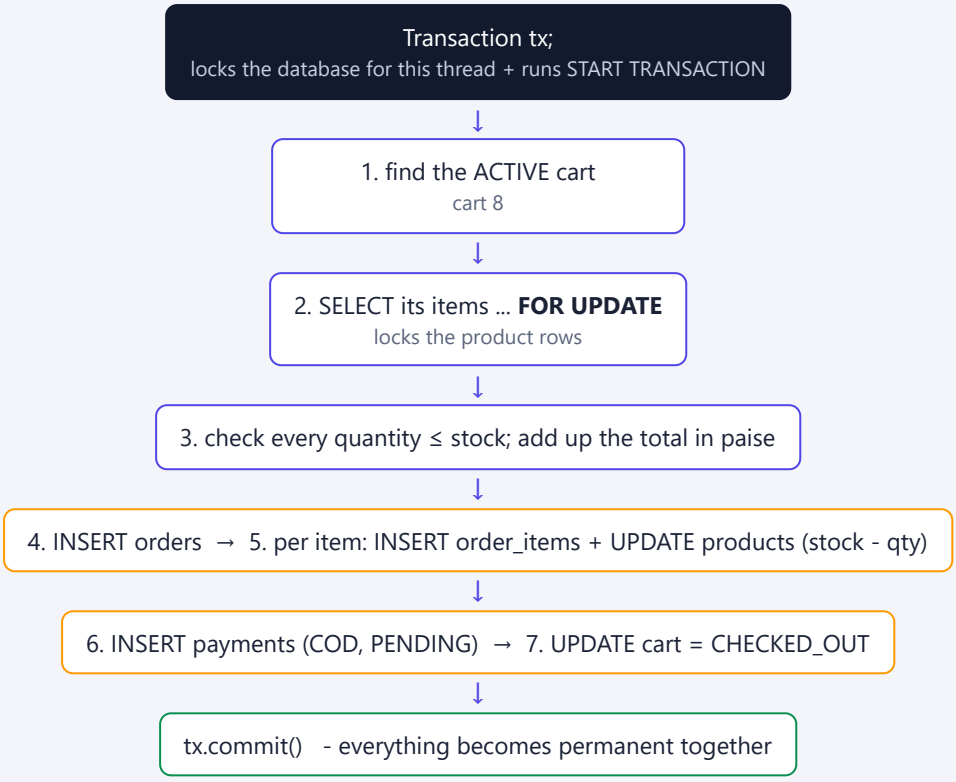


Figure 6 - checkout is all-or-nothing.

The exact statements, with this example's values (cart 8, user 5, 2 × ANC Headphones):

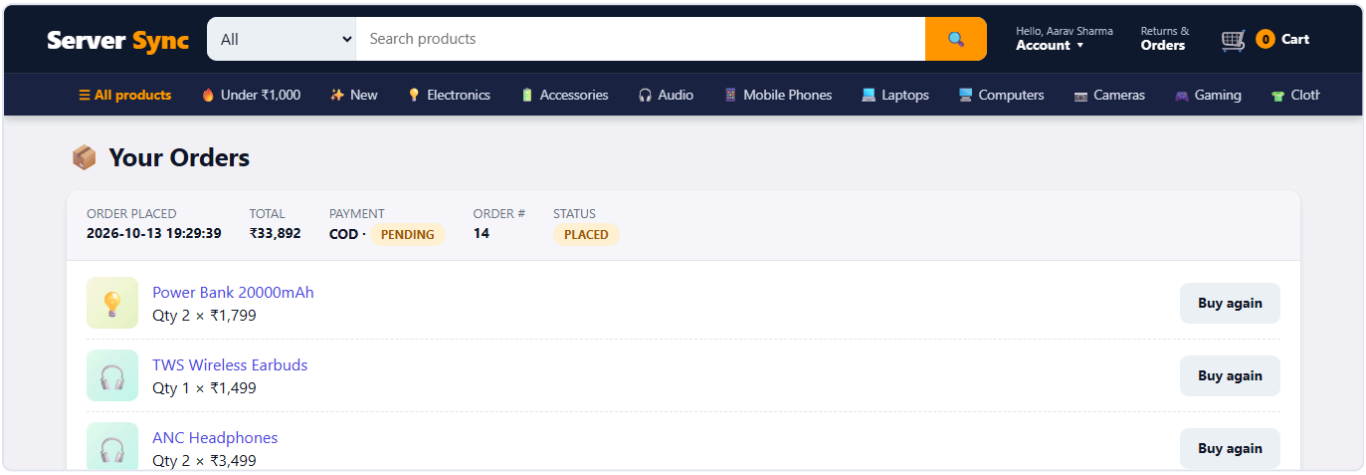
```
START TRANSACTION;
SELECT cart_id FROM cart WHERE user_id=5 AND status='ACTIVE' ORDER BY cart_id DESC LIMIT 1;        -- 8
SELECT ci.product_id, ci.quantity, p.price, p.stock
  FROM cart_items ci JOIN products p ON ci.product_id=p.product_id WHERE ci.cart_id=8 FOR UPDATE;  -- (11, 2,
3499.00, 15)
-- C++: qty 2 <= stock 15 OK.   total = 2 * 349900 paise = 699800 paise  ->  "6998.00"
INSERT INTO orders (user_id, cart_id, total, status, deliver_at) VALUES (5, 8, 6998.00, 'PLACED',
DATE_ADD(NOW(), INTERVAL 87 SECOND));  -- order id 9, auto-delivery in 87 s (random 60-120)
INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (9, 11, 2, 3499.00);       -- price
copied!
UPDATE products SET stock=stock-2 WHERE product_id=11 AND stock>=2;                                -- 1 row
changed
INSERT INTO payments (order_id, amount, method, status) VALUES (9, 6998.00, 'COD', 'PENDING');
UPDATE cart SET status='CHECKED_OUT' WHERE cart_id=8;
COMMIT;
```

Table	Before	After COMMIT
`products` row 11	stock = 15	stock = **13**
`orders`	-	new row #9: total 6998.00, status `PLACED`
`order_items`	-	(order 9, product 11, qty 2, price 3499.00)
`payments`	-	(order 9, 6998.00, `COD`, `PENDING`)
`cart` 8	`ACTIVE`	`CHECKED_OUT` (so the next "Add to cart" opens a fresh cart)

What "all-or-nothing" means in practice

Imagine the `INSERT INTO payments` failed. The function does `out.error = pay.error; return out;` - it returns **without** calling `tx.commit()`. When `tx` goes out of scope its destructor runs `ROLLBACK`, so the order, the order item **and the stock reduction** all vanish. No half-finished orders, no missing stock.

Finally the handler redirects to `/orders?notice=ordered`; `myOrdersPage()` lists the order with its items and the badges PLACED / PENDING.



/orders after the order is placed.

Part F - Example 4: the admin cancels an order

F1. Becoming an admin

There is deliberately no "register as admin" button. Instead you sign up normally and run, in a second terminal:

```
.\server.exe --make-admin you@email.com
```

This starts the *same* program, but `main()` sees `argv[1] == "--make-admin"` after step 4 (config → DB → migrate → categories), calls `user_service.cpp` `makeAdminByEmail()` (`UPDATE users SET role='ADMIN' WHERE email='...'`), prints a message and **exits without starting the web server**. After signing in again, `userForSession()` returns `role = "ADMIN"`, so `req.user.isAdmin()` is true and the header gains an *Admin* link.

F2. Opening the order

`GET /admin/order?id=9` → `r.get("/admin/order", requireAdmin(adminOrderDetail))`. The wrapper checks the role first; a customer would be redirected to `/?notice=forbidden`. `adminOrderDetail()` (`handlers/admin_orders.cpp`) calls `getOrder(9)` and `orderItems(9)`, builds `Data`, and renders `admin/order_detail.html` inside `admin/layout.html` (the sidebar layout) via `renderAdminPage()`.

F3. Clicking "Update status" with CANCELLED

```
POST /admin/order/status
id=9&status=CANCELLED
```

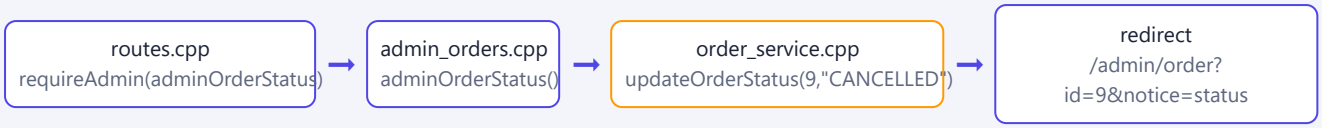


Figure 7 - control flow for an admin action.

`updateOrderStatus()` is another transaction. In order:

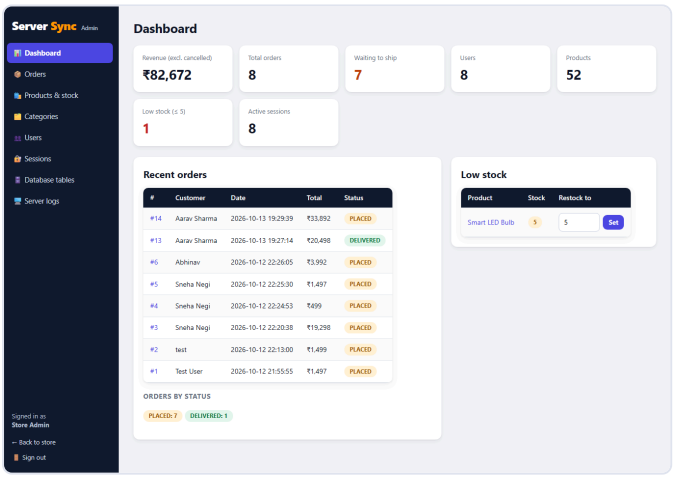
1. `Transaction tx;` - lock + `START TRANSACTION`.
2. `getOrder(9)` - current status is `PLACED`, payment `PENDING`.
3. **Business rules** (all enforced in code, not in the HTML):
   - same status → nothing to do;
   - `DELIVERED → CANCELLED` → refused ("a delivered order cannot be cancelled");
   - already `CANCELLED` → refused (its stock was already returned, so reopening would corrupt stock counts).
4. Because the new status is `CANCELLED`:

```
UPDATE products p JOIN order_items oi ON oi.product_id=p.product_id
   SET p.stock = p.stock + oi.quantity WHERE oi.order_id=9;     -- product 11: 13 -> 15
UPDATE payments SET status='CANCELLED' WHERE order_id=9;       -- 'REFUNDED' if it had been PAID
```

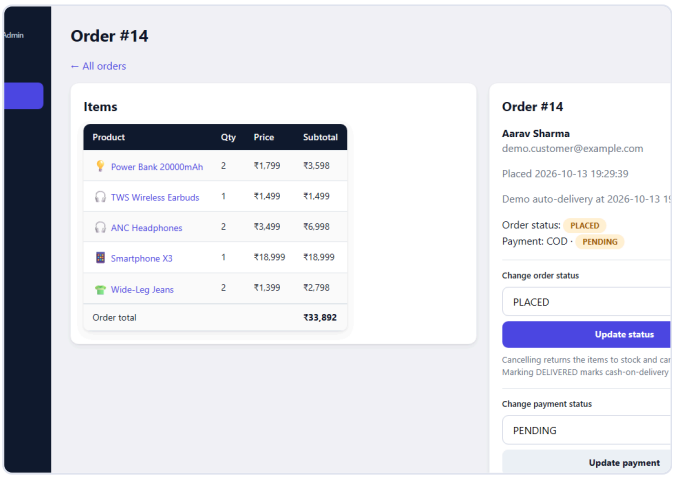
5. `UPDATE orders SET status='CANCELLED' WHERE order_id=9`, then `tx.commit()`.

The handler redirects with `notice=status`, the page reloads showing "*Order status updated*", the order now has a red CANCELLED badge, and the status form is replaced by "*This order was cancelled and can no longer be changed.*" The shopper's `/orders` page shows the same status because both read the same `orders` table.

Marking an order DELIVERED follows the same path but runs `UPDATE payments SET status='PAID' ... WHERE method='COD' AND status='PENDING'` - cash is collected on delivery.



/admin - numbers come from `dashboardStats()` (one SQL with sub-selects).



/admin/order?id=... - change order or payment status.

F4. The other admin actions use the same recipe

Admin action	URL	Handler (handlers/)	Service function	What changes
Set stock	POST /admin/products/stock	admin_catalog	`setProductStock`	`UPDATE products SET stock=?`

Add / edit product	POST /admin/products/save	admin_catalog	`createProduct` / `updateProduct` (validate first)	`INSERT / UPDATE products`
Delete product	POST /admin/products/delete	admin_catalog	`deleteProduct` (refuses if in past orders)	removes cart rows, then the product
Add / rename / delete category	POST /admin/categories/...	admin_catalog	`createCategory ...` then `loadCategories()`	categories table + in-memory menu
Change user role	POST /admin/users/role	admin_users	`setUserRole` (cannot change own role)	`UPDATE users SET role`
End a session	POST /admin/sessions/end	admin_users	`endSession`	`is_active = FALSE`
Browse a table	GET /admin/table?name=orders	admin_dashboard	`readTable` (name must be on a whitelist)	read-only; password column masked

Part G - The template engine, line by line

`core/template.cpp` is about 200 lines and is why no HTML appears in the C++ code. A handler fills a `Data`; the engine pours it into an HTML file.

The data structure

```
struct Data {
    map<string, string>       values;   // "name" -> "ANC Headphones"
    map<string, vector<Data>> lists;    // "products" -> [ Data, Data, Data ... ]
};
```

A `Data` can contain lists of other `Data` objects, so an order can contain its items, and a table can contain rows that contain cells.

The five tags

In the HTML file	What the engine does
`{{name}}`	Look up `name`, **escape** `& < > " '`, insert. This is the XSS protection: a product named `<script>` is displayed, not executed.
`{{{content}}}`	Insert as-is (used only for HTML that the server itself built, such as the page body inside the layout).
`{{#if x}} A {{else}} B {{/if}}`	True if `x` is non-empty and not "0", or is a non-empty list.
`{{#each list}} ... {{/each}}`	Repeat the inner text once per item, with that item's values in scope.
`{{> partials/x.html}}`	Read another template file and render it with the current data.

How it works inside

1. **renderTemplate(file, data)**
   Reads `frontend/templates/<file>` from disk (every time - so editing HTML needs only a browser refresh, not a rebuild) and starts `renderText` with a **Scope**.

2. **renderText scans for `{{`**
   Text before a tag is copied to the output unchanged. At a tag it decides: raw `{{{ }}}`, include `{{>`, `{{#if`, `{{#each`, or a plain variable.

3. **Blocks: findBlockEnd()**
   For `#if` / `#each` it must find the *matching* closing tag even when blocks are nested. It counts: every opening tag adds 1, every closing tag subtracts 1, and the one that brings the count to 0 is the match. `findElse()` does the same to locate the `{{else}}` belonging to this `if` and not to a nested one.

4. **Scope chain**
   A `Scope` is `{ current Data, pointer to parent Scope }`. Inside `#each` a new scope is created for every item whose parent is the page scope. `lookupValue()` walks outwards: item first, then the page. That is why the product card can use both its own `{{name}}` and the page-wide `{{loggedIn}}`.

A tiny trace

Handler data: `heading="Audio"`, `loggedIn=""` (guest), and a list `products` with two items.

```
<h1>{{heading}}</h1>
{{#each products}}
  <div>{{name}} - {{price}}
  {{#if loggedIn}} <button>Add</button> {{else}} <button>Sign in to buy</button> {{/if}}</div>
{{/each}}
```

```
<h1>Audio</h1>
  <div>Neckband Earphones - ₹999     <button>Sign in to buy</button></div>
  <div>TWS Wireless Earbuds - ₹1,499 <button>Sign in to buy</button></div>
```

`name` and `price` were found in the item's own scope; `loggedIn` was not there, so the lookup moved to the parent (page) scope and found it empty → the `else` branch.

Layout nesting
`renderPage()` renders the *page* template first into a string, stores it as `content`, then renders `layout.html`, which contains `{{{content}}}`. The admin panel does the same with `admin/layout.html`. Changing the header for the whole site = editing one file, `partials/header.html`.

Part H - Two shoppers, one last item

Concurrency is where many projects fail. The scenario: **stock of "ANC Headphones" is 1**. Ravi (worker 1) and Meera (worker 2) both press *Place order* at the same instant.

Time	Worker 1 - Ravi	Worker 2 - Meera
t1	`Transaction tx` → takes the database lock, `START TRANSACTION`	`Transaction tx` → tries to take the lock... **waits**
t2	reads cart: qty 1, stock 1 → OK	(waiting)
t3	inserts order, order item; `UPDATE products SET stock=stock-1 WHERE stock>=1` → stock 0	(waiting)
t4	`COMMIT` ; lock released	lock acquired, `START TRANSACTION`
t5	redirect to /orders - success	reads cart: qty 1, **stock 0** → "Not enough stock for one or more items" → returns, **rollback**
Result	Exactly one order exists; stock is 0, never negative.	

The three layers of protection

Layer	Where	What it does
1. C++ lock	`DbLock` / `Transaction` in `db/database.h`	Only one thread uses the single MySQL connection at a time; a checkout holds it from `START TRANSACTION` to `COMMIT` .
2. Row lock	`SELECT ... FOR UPDATE` in `placeOrder()`	MySQL itself locks those product rows until commit. With today's single connection layer 1 already serialises everything, but this becomes essential the day you add a connection pool.
3. Guarded update	`UPDATE ... WHERE stock >= qty` + "exactly 1 row changed"	Even if both earlier layers failed, the database refuses to go below zero and checkout aborts.

Why a *recursive* mutex?

`Transaction` already holds the lock; inside it, `placeOrder()` calls `activeCartId()` , which does `DbLock lock;` again, and then `db.run()` , which locks once more. With an ordinary mutex the same thread would deadlock against itself. A `std::recursive_mutex` lets the **same** thread lock repeatedly while still blocking **other** threads.

> **Honest limitation**
> The single shared connection serialises *all* database work, so the 4 workers cannot run queries truly in parallel. That is simple and safe for a coursework project; the upgrade path is one connection per worker (a pool), which is why layers 2 and 3 are already in place.

Test it yourself

Set a product's stock to 1 in Admin → Products. Put it in the carts of two accounts. Submit both checkouts at the same moment (two terminals with `curl`, or two browser tabs). Expect one success, one "Not enough stock" message, and stock = 0.

Part I - Every file: when it runs, who calls it

backend/ (top level)

File	When it runs	Calls / is called by
`main.cpp`	First, at program start	Calls config, db, migrate, category_service, user_service (`--make-admin`), `runServer` .
`config.cpp / .h`	Step 1 of start-up; values read all the time	Used by database.cpp (DB login), template.cpp (frontend folder), app.cpp, admin_service.
`routes.cpp`	Before `main()` (via `app.cpp` 's static router)	Binds every URL to a handler; applies `requireLogin/Admin` .
`app.cpp`	Every request (the callback given to `runServer`)	Calls serveStatic, `userForSession` , `cartItemCount` , router, `notFoundPage` .

backend/core/ - knows the web, not the shop

File	When it runs	Job
`server.cpp`	Start-up (sockets, threads), then forever	Accept loop, worker threads, read the request, send the response, close.
`http.cpp`	Twice per request	`parseRequest` (in) and `buildHttpResponse` (out); cookies; `urlDecode/urlEncode` .
`router.cpp`	Once per request	Look up `"METHOD /path"` ; `requireLogin` , `requireAdmin` .
`template.cpp`	Once or more per page	Merge `Data` into HTML files.

backend/db/ and backend/services/ - knows the shop and the SQL

File	When it runs	Job
`db/database.cpp`	Start-up, then every query	`connect` , `run/rows/quote` , `Transaction` , `migrateDatabase` .
`user_service.cpp`	Sign in/up, every authenticated request, admin pages	Password hashing, sessions, roles.
`product_service.cpp`	Home, product pages, admin products	Search/filter/sort SQL; create, edit, stock, delete with validation.

`category_service.cpp`	Start-up; menu on every page; admin categories	Cached list; create, rename, delete.
`cart_service.cpp`	Header badge on every request; cart actions	Cart lines, add/update/remove with stock checks.
`order_service.cpp`	Checkout, order lists, admin order actions	`placeOrder` transaction; status changes; stock restore on cancel.
`admin_service.cpp`	Admin dashboard and table browser	KPI numbers, whitelisted table viewer.

backend/handlers/ - one function per page

File	Handlers inside
`shop_handlers.cpp`	`homePage, productsPage, productDetailPage`
`auth_handlers.cpp`	`loginPage, loginSubmit, signupPage, signupSubmit, logout`
`cart_handlers.cpp`	`cartPage, cartAdd, cartUpdate, cartRemove`
`order_handlers.cpp`	`checkoutPage, placeOrderSubmit, myOrdersPage, mySessionsPage`
`admin_dashboard.cpp`	`adminDashboard, adminTables, adminTableView`
`admin_catalog.cpp`	products: `adminProducts, adminProductForm, adminProductSave, adminProductStock, adminProductDelete`; categories: `adminCategories, adminCategorySave, adminCategoryDelete`
`admin_orders.cpp`	`adminOrders, adminOrderDetail, adminOrderStatus, adminOrderPayment`
`admin_users.cpp`	`adminUsers, adminUserRole, adminSessions, adminSessionEnd`
`view.cpp`	Shared helpers: `renderPage, renderAdminPage, messagePage, notFoundPage, money, productData, addProducts, statusClass, categoryIcon/Tint, adminBackUrl`, and the flash-message table.
`handlers.h`	Declares every handler in one list (a table of contents for the handlers).

frontend/

File(s)	Used for
`templates/layout.html`	Shell of every storefront page: header, flash bar, `{{{content}}}`, footer, guest pop-ups.
`templates/partials/`	`header.html`, `footer.html`, `guest_modals.html`, `product_card.html` (reused on home, list, detail, related).
`templates/pages/`	`home, products, product, cart, checkout, orders, sessions, login, signup, message`

`templates/admin/`	`layout` (sidebar), `dashboard, products, product_form, categories, orders, order_detail, users, sessions, tables, table_view, message`
`static/css/style.css, admin.css`	Look and feel. Served by `app.cpp` 's `serveStatic` .
`static/js/app.js`	Login pop-up open/close, quantity +/- buttons, auto-hiding message bar.

database/

`ecommerce_db.sql` (tables, 15 categories, 52 products) and `verify_and_seed.sql` are run by **you** in MySQL Workbench; the server never reads them. The only schema change the server makes itself is `users.role` .

Part J - Call-chain cheat sheet

Every feature has the same skeleton: **server** → **http** → **app** → **router/routes** → **handler** → **service** → **database** → **(template)** → **http** → **server**. The interesting middle of each:

Feature	Chain (middle part)
Browse / filter products	`routes.cpp` → `productsPage` → `findProducts` → `db.rows` → `productData` → `renderPage` → `pages/products.html` + `partials/product_card.html`
Product detail	`productDetailPage` → `getProduct` , `relatedProducts` → `pages/product.html`
Sign up	`signupSubmit` → `signUp` (check duplicate, hash, `INSERT users`) → redirect `/login?notice=signup`
Sign in	`loginSubmit` → `logIn` → `startSession` → redirect + `Set-Cookie`
Who am I? (every request)	`app.cpp identifyUser` → `userForSession` → `cartItemCount`
Sign out	`logout` → `endSession` → clear cookie → redirect
Add to cart	`requireLogin` → `cartAdd` → `addToCart` → redirect `/cart`
View cart	`cartPage` → `cartItems` → `pages/cart.html`
Place order	`placeOrderSubmit` → `placeOrder` (`Transaction`) → redirect `/orders`
Order history	`myOrdersPage` → `listOrders` + `orderItems` → `pages/orders.html`
Admin dashboard	`requireAdmin` → `adminDashboard` → `dashboardStats` , `ordersByStatus` , `listOrders` , `findProducts(maxStock)` → `admin/dashboard.html`
Update stock	`adminProductStock` → `setProductStock` → redirect back (only to `/admin...`)
Add product	`adminProductSave` → `createProduct` (`validate`) → redirect, or re-show form with the error
Cancel / ship / deliver	`adminOrderStatus` → `updateOrderStatus` (`Transaction`) → redirect
Static file	`app.cpp serveStatic` → read `frontend/static/...` → send (no handler, no DB)

Unknown URL	`router.dispatch` returns false → `notFoundPage` (HTTP 404)

Why most actions end with a redirect

After any **POST** that changes data, the handler returns `Response::redirect(...)` instead of a page. The browser then makes a harmless **GET** to show the result. If the user presses Refresh they repeat the GET, not the POST - so an order can never be placed twice by accident. The `?notice=code` on the redirect URL is how the success message travels to the next page.

Part K - Where to look when something breaks

Symptom	Look at
Program exits immediately / "Connection failed"	MySQL running? `ECOM_DB_PASS`? → `config.cpp`, `database.cpp` `connect()`
"Bind failed"	Another `server.exe` owns port 8080 → `taskkill /F /IM server.exe`
Page shows `missing template: ...`	Started from the wrong folder, or the file name in the handler is misspelled → `config::load()`, the handler's `renderPage(...)` string
Page loads but is unstyled	`/static/css/style.css` request failing → `app.cpp serveStatic`, `frontendDir`
A page is 404	No line for that *method* + *path* in `routes.cpp` (a link is GET but the route is POST?)
Redirected to login unexpectedly	Cookie missing/expired or session ended → `userForSession`, `user_sessions.is_active`
Redirected to `?notice=forbidden`	Account is not ADMIN → run `server.exe --make-admin email`, sign in again
Wrong data on a page	Read the SQL in the service; run it in MySQL Workbench. Then check the `Data` key name in the handler against `{{name}}` in the template
A `{{value}}` shows empty	Key not set (typo) or set in a different scope (inside `#each` use the item's keys)
Stock numbers look wrong	`placeOrder` (reduces), `updateOrderStatus` (restores on cancel), admin `setProductStock` (overrides)
Build error `mysql.h not found`	Edit `MYSQL=` path in `build.bat`
Console shows what happened	Each request prints `[Wn] METHOD /path`; failed queries print `[DB] Query failed: ...`

Adding a page - the four-step recipe

Example: a wishlist page.

1. **Service** - `services/wishlist_service.cpp` : `Rows listWishlist(int userId)` with the SQL.
2. **Handler** - `Response wishlistPage(Request& req)` : call the service, fill a `Data` , `return renderPage(req, "Wishlist", "pages/wishlist.html", page);` and declare it in `handlers.h` .
3. **Template** - `frontend/templates/pages/wishlist.html` using `{{#each items}}` .
4. **Route** - one line in `routes.cpp` : `r.get("/wishlist", requireLogin(wishlistPage));`

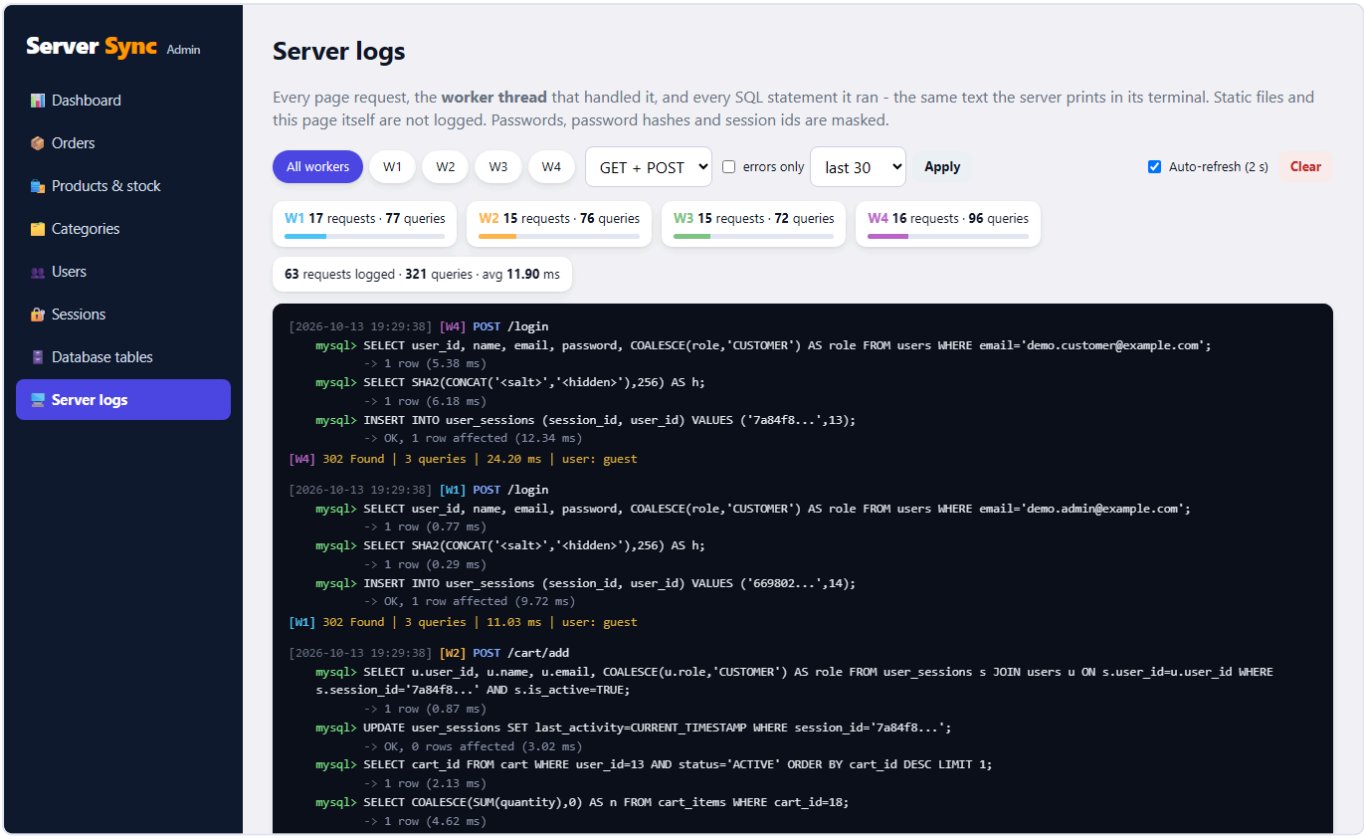
Run `build.bat` (new `.cpp` files are picked up automatically) and restart `server.exe` .

Glossary

Term	Meaning here
Socket	An endpoint of a network connection; the operating system hands the program one per browser connection.
HTTP request / response	The text a browser sends / the text the server returns (status line, headers, blank line, body).
GET / POST	GET reads a page; POST submits a form that *changes* something.
Route	The rule "this method + this path → this handler".
Handler	A C++ function `Response f(Request&)` that produces one page or performs one action.
Service	A group of functions that implement business rules and own the SQL.
Template	An HTML file with `{{ }}` placeholders filled by a `Data` object.
Session / cookie	A random id stored in the database and in the browser; links later requests to a logged-in user.
Redirect (302)	"Go to this other URL" - used after POST (Post/Redirect/Get).
Transaction	A group of SQL statements that either all succeed (COMMIT) or all are undone (ROLLBACK).
Mutex / recursive mutex	A lock so only one thread uses shared data at a time; the recursive kind lets the same thread lock again.
Thread pool	A fixed set of worker threads that take jobs from a queue.
SQL injection / XSS	Attacks via crafted input in SQL / in HTML. Prevented by `db.quote()` and template escaping.

Part L - Watching it live: the Server logs page

Everything in Parts A-K happens invisibly. **Admin → Server logs** (`/admin/logs`) makes it visible: for every request it shows **which worker thread handled it** and **every SQL statement that request ran**, in terminal style. (The real terminal, for example the VS Code terminal, now only shows the worker for each request, such as `[W2] GET /products` ; the SQL lives here.) It is the quickest way to *see* the thread pool and the transaction from Part E working.



/admin/logs - coloured worker ids (W1-W4), per-worker request/query counts, and the SQL of each request.

What a log entry looks like

```
[2026-10-13 18:48:35] [W3] POST /checkout/place           <- time, worker, method, URL
    mysql> START TRANSACTION;
           -> OK, 0 rows affected (0.32 ms)
    mysql> SELECT ci.product_id, ci.quantity, p.price, p.stock FROM cart_items ci JOIN products p ... FOR
UPDATE;
           -> 1 row (0.67 ms)
    mysql> UPDATE products SET stock=stock-1 WHERE product_id=2 AND stock>=1;
           -> OK, 1 row affected (1.92 ms)
    mysql> COMMIT;
           -> OK, 0 rows affected (12.55 ms)
[W3] 302 Found | 13 queries | 32.77 ms | user: Log Tester (admin)   <- result line
```

How it works - the files involved

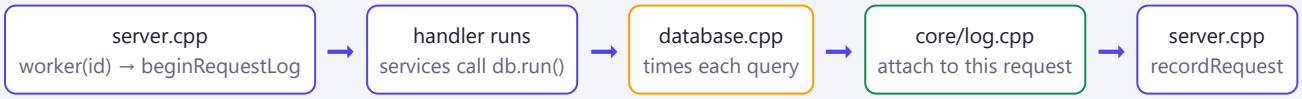


Figure L1 - how queries find their request.

1 The worker starts a log for the request

`core/server.cpp`

```
RequestLog log;                        // worker id, method, URL, time ...
log.worker = id;                       // the number this worker thread was started with
beginRequestLog(&log);                 // remember "the current request" - per thread
res = handler(req);                    // ... all the work of Parts C-F happens here ...
endRequestLog();
recordRequest(log);                    // store it and print it
```

Static files (`/static/...`) and the log pages themselves are skipped - otherwise viewing the log would fill the log.

2 Every query reports itself

`db/database.cpp` `Database::run()`

```
auto started = steady_clock::now();
Result result = execute(sql);          // the real mysql_query call
double ms = elapsed since 'started';
addQueryToCurrentRequest(sql, ms, result.ok, result.error, isSelect, rowCount);
```

`addQueryToCurrentRequest` uses a **thread-local** pointer (`thread_local RequestLog* currentLog`). Each worker thread has its own copy, so Worker 1's queries can never land in Worker 2's request - even when both run at the same instant. Queries run at start-up (no request) are ignored.

3 Cleaning before storing

`core/log.cpp` `cleanSqlForLog()`

The SQL is collapsed to one line and **secrets are masked**: the password-hashing query becomes `SHA2(CONCAT('<salt>','<hidden>'),256)`, password hashes become `'<password-hash>'`, and 32-character session ids become `'67f805...'`. Form bodies are never logged. This matters because an admin (or anyone watching the console) must not be able to read passwords or hijack sessions from a log.

4 Store and print

`core/log.cpp` `recordRequest()`

The finished request goes into a **ring buffer** (`deque`, newest 300 kept, guarded by a mutex) for the admin page. The terminal gets just one short line, `[W2] GET /products`, written under a mutex so lines from different workers never interleave. Set `ECOM_LOG_SQL=1` if you also want the SQL printed in the terminal; the admin page always has the full log.

5 The page

`handlers/admin_logs.cpp` `admin/logs.html, logs_feed.html, static/js/logs.js`

`adminLogs()` calls `recentRequests(worker, method, errorsOnly, limit)`, counts requests/queries per worker, and fills a `Data`. The terminal panel is its own template (`logs_feed.html`); with **Auto-refresh** ticked, `logs.js` fetches `/admin/logs/feed` every 2 seconds and swaps the panel without reloading the page. Filters: worker (W1-W4), GET/POST, errors only, last 30/60/150/300.

Things to try (good for demonstrations)

1. **See the thread pool:** open Server logs, then in a terminal run `for i in 1 2 3 4 5 6 7 8; do curl -s -o /dev/null "localhost:8080/products?q=phone" & done` - the eight requests are spread over W1-W4 and the per-worker counters rise together.
2. **Watch a transaction:** place an order as a customer and find `POST /checkout/place` - you will see `START TRANSACTION` ... `COMMIT` around the `INSERT`s and the stock `UPDATE`.
3. **See a rollback:** make a query fail (for example place an order for more than the stock) and tick *errors only*; the failed statement shows `-> ERROR: ...` in red.
4. **Measure:** compare the `ms` of the same page before and after you change a query; the footer line shows total time per request.

Limits to mention

The log lives in memory (lost when the server stops) and keeps only the last 300 requests. The log records the URL query string as sent, so do not put secrets in URLs. It is a teaching and debugging tool, not an audit trail.

Part M - Demo delivery and customer cancel (rollback)

For demonstrations the shop simulates delivery. Checkout offers **only Cash on Delivery**. After you place an order a countdown starts; when it reaches zero the order becomes **DELIVERED** and the cash payment becomes **PAID** - without anyone clicking anything. Until then the customer can **cancel**, and the cancel undoes the order's effects.



Figure M1 - the two possible endings of a demo order.

1 1. Scheduling - at checkout

`services/order_service.cpp` `placeOrder()`

A new column `orders.deliver_at` (added automatically at start-up by `migrateDatabase()`) is filled with `DATE_ADD(NOW(), INTERVAL n SECOND)`, where `n` is random between 60 and 120 (`config::deliveryMinSeconds/MaxSeconds`). Old orders keep `deliver_at = NULL` and are never auto-delivered.

2. The delivery thread

`main.cpp` → `order_service.cpp` `deliverDueOrders()`

`main()` starts one extra background thread (a sixth thread next to the accept loop and the four workers) that wakes every 5 seconds. In one transaction it runs:

```
UPDATE payments p JOIN orders o ON o.order_id=p.order_id SET p.status='PAID' ...
  WHERE o.status IN ('PLACED','SHIPPED') AND o.deliver_at<=NOW() AND p.method='COD' AND
p.status='PENDING';
UPDATE orders SET status='DELIVERED' WHERE status IN ('PLACED','SHIPPED') AND deliver_at IS NOT NULL
AND deliver_at<=NOW();
```

3. The countdown in the browser

`order_handlers.cpp` → `pages/orders.html` , `static/js/app.js`

`listOrders()` returns `seconds_left` (`TIMESTAMPDIFF(SECOND, NOW(), deliver_at)`). The template prints it in `<b class="countdown" data-seconds="72">` ; JavaScript counts down each second and, at zero, reloads the page after 6 seconds - by then the thread has marked the order delivered, so the page shows "*Delivered - cash collected on delivery*".

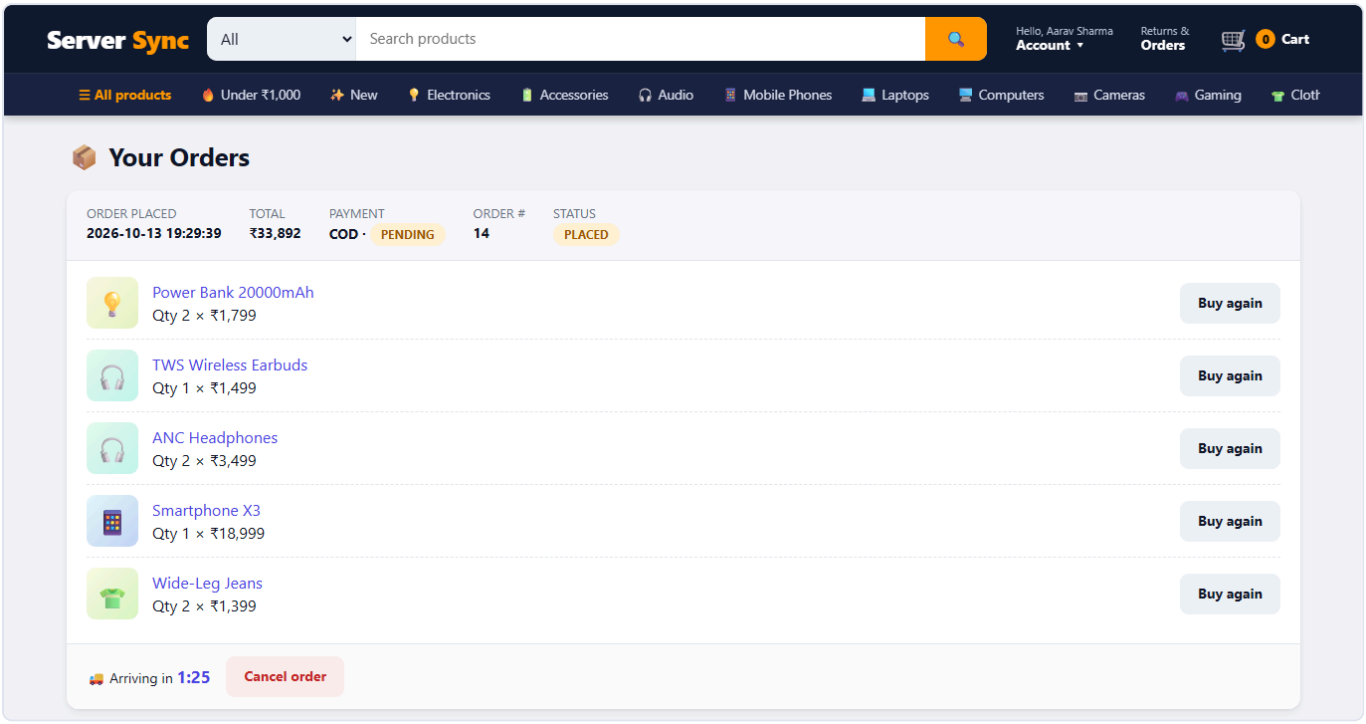
4. Cancelling - the rollback

`order_handlers.cpp` `cancelMyOrder()` → `order_service.cpp` `cancelOrderByCustomer()` → `updateOrderStatus()`

1. Route: `r.post("/orders/cancel", requireLogin(cancelMyOrder));`
2. `cancelOrderByCustomer(userId, orderId)` checks the order **belongs to this customer** ("not found" otherwise - nobody can cancel someone else's order) and that it is not already CANCELLED or DELIVERED ("Too late...").
3. `updateOrderStatus(orderId, "CANCELLED")` opens a `Transaction` and **checks the status again inside it**, then runs:

   ```
   UPDATE products p JOIN order_items oi ON oi.product_id=p.product_id SET p.stock = p.stock +
   oi.quantity WHERE oi.order_id=14;  -- stock back
   UPDATE payments SET status='CANCELLED' WHERE order_id=14;            -- 'REFUNDED' if it had been
   PAID
   UPDATE orders SET status='CANCELLED' WHERE order_id=14;
   COMMIT;
   ```

4. The customer is redirected to `/orders?notice=cancelled` .



/orders - live countdown ("Arriving in 1:25") and the Cancel order button.

The race: cancel at the exact moment of delivery

The delivery thread and a customer's cancel may fire at the same instant. Both work inside a `Transaction` (which holds the C++ database lock) and both check the order's **current status inside the transaction**. So one runs first and the other sees the result: if the delivery wins, the cancel is refused with "*Too late - this order has already been delivered*"; if the cancel wins, the order is CANCELLED and the delivery query (`WHERE status IN ('PLACED','SHIPPED')`) no longer matches it. A delivered order is never cancelled, and a cancelled one is never delivered.

Speeding the demo up

Start the server with `ECOM_DELIVERY_MIN=15` and `ECOM_DELIVERY_MAX=20` (PowerShell: `$env:ECOM_DELIVERY_MIN=15; $env:ECOM_DELIVERY_MAX=20; .\server.exe`) for 15-20 second deliveries. Admins can still change an order's status by hand in Admin → Orders.

Remember the whole thing in one sentence

A worker thread reads a request, `app.cpp` finds the user, `routes.cpp` picks a handler, the handler asks a service (which runs SQL through `database.cpp`) for data, fills a `Data` object, and a template turns it into the page that the worker sends back.